Ein Testverfahren und die konstruktive Erzeugung von Bitfolgen mit exponentiell kumulierten Häufigkeitsverteilungen ihrer Läufe

Auswertung endlicher Bitfolgen

Nicolas Windt

28. Oktober 2026

Wir führen den *Einschritttest* ein, ein Auswertungsverfahren für endliche Bitfolgen, das die Häufigkeiten der Läufe und Teilläufe einer Folge kumuliert. Die Anwendung auf die Ausgabe des Zufallsgeräts `/dev/random` einer Linux-Installation zeigt einen annähernd exponentiell fallenden Verlauf der kumulierten Häufigkeiten, im Einklang mit der geometrischen Verteilung der Lauflängen unabhängiger Bits. Motiviert durch diese Beobachtung untersuchen wir Bitfolgen, deren kumulierte Häufigkeitsverteilung exakt exponentiell fällt, und fassen sie zu Klassen B_N zusammen. Wir zeigen, dass die Verteilung der Läufe ohne Teilläufe (Grundverteilung) der Elemente aus B_N einer strengen Rekursion genügt, und leiten eine Identität für die Anzahl der abgegoltenen Teilläufe her. Darauf aufbauend geben wir einen iterativen und einen rekursiven Algorithmus zur Erzeugung eines Elements aus B_N an. Schließlich ordnen wir jeder natürlichen Zahl eine eindeutige Zusammensetzung paarweise verschiedener Klassen B_N zu, erhalten daraus eine Familie zusammengesetzter Häufigkeitsverteilungen und diskutieren den dadurch induzierten Zufallsbegriff.

Schlüsselwörter: Zufallsfolge · Lauf · Runs-Test · kumulierte Häufigkeitsverteilung · exponentielle Verteilung · Zufallszahlengenerator · `/dev/random`

1. Einleitung

Ausgangspunkt dieser Arbeit sind Karl Poppers Untersuchungen zum Wahrscheinlichkeitsbegriff in „Logik der Forschung“ [8]: Zufälligkeit ist keine Eigenschaft, die sich an einem einzelnen Ereignis ablesen lässt, sondern eine Zuschreibung, die an der Wiederholbarkeit und Überprüfbarkeit von Beobachtungen hängt. Für die hier behandelten endlichen Bitfolgen ziehen wir daraus die Konsequenz, den Zufallsbegriff nicht axiomatisch vorauszusetzen, sondern endliche Folgen anhand beobachtbarer, rein kombinatorischer Strukturen zu untersuchen – ohne Rückgriff auf Grenzwertaussagen über unendliche Folgen.

In dieser Arbeit wird zuerst ein Auswertungsverfahren für endliche Bitfolgen entworfen, der Einschritttest (Abschnitt 4), und das Ergebnis seiner Anwendung auf die Ausgabe eines Zufallsgeräts analysiert (Abschnitt 5). Das Experiment wird zeigen, dass sich die Häufigkeitsverteilung der Läufe und Teilläufe der Ausgabe eines Zufallsgenerators einer exponentiell gearteten Verteilung annähert. Motiviert durch dieses experimentelle Ergebnis wird die Struktur der Häufigkeitsverteilungen solcher Bitfolgen formal untersucht und damit die experimentelle Beobachtung erklärt (Abschnitt 6).

Schließlich werden Algorithmen angegeben, die Elemente der dabei auftretenden Klassen erzeugen (Abschnitt 7), und es wird gezeigt, wie sich diese Klassen über die binäre Darstellung natürlicher Zahlen zusammensetzen lassen (Abschnitt 8). Abschnitt 9 diskutiert die Konsequenzen für den Zufallsbegriff, Abschnitt 10 fasst zusammen.

Die Beiträge im Einzelnen:

- ein lineares Auswertungsverfahren, das die Häufigkeiten von Läufen *und* Teilläufen endlicher Bitfolgen kumuliert (Einschritttest, Abschnitt 4);
- eine experimentelle Auswertung der Ausgabe von `/dev/random` (Abschnitt 5);
- die Klassen B_N der Bitfolgen mit exakt exponentiell fallender kumulierter Häufigkeitsverteilung, inklusive der Struktur ihrer Grundverteilung (Satz 12) und einer Abgeltungsidentität für Teilläufe (Lemma 17);
- ein iterativer und ein rekursiver Generator für Elemente aus B_N (Abschnitt 7);
- eine eineindeutige Zuordnung zwischen natürlichen Zahlen und Zusammensetzungen paarweise verschiedener Klassen B_N (Abschnitt 8).

2. Stand der Forschung

Die mathematische Grundlage des Zufallsbegriffs ist die Kolmogoroff-Axiomatik [4]; der daraus folgende starke und schwache Grenzwertsatz begründet die Erwartung, dass in hinreichend langen Folgen unabhängiger, gleichverteilter Bits die Lauflängen geometrisch verteilt sind: Die Wahrscheinlichkeit einer Lauflänge k beträgt 2^{-k} [1, und dortige Übungsaufgaben zu Runs]. Der hier beobachtete exponentielle Abfall der (kumulierten) Häufigkeiten ist daher für ideale Zufallsfolgen zu erwarten; neu in dieser Arbeit ist nicht der Abfall selbst, sondern die Kumulation über die Teilläufe sowie die Untersuchung derjenigen endlichen Folgen, deren kumulierte Verteilung *exakt* exponentiell fällt.

Statistische Verfahren zur Prüfung von Zufallsfolgen sind seit langem etabliert. Der klassische Runs-Test von Wald und Wolfowitz [10] prüft die Anzahl der Läufe einer Folge gegen ihre erwartete Verteilung; er ist bis heute Bestandteil der standardisierten Testbatterien für kryptographische Zufallsgeneratoren, etwa der NIST-Suite [9] und ihrer Vorläufer [3, 6]. Übersichten über den Entwurf und die Prüfung von Zufallszahlengeneratoren geben Knuth [3] und L'Ecuyer [5]. Der algorithmische Zufallsbegriff von Martin-Löf [7] definiert Zufälligkeit unendlicher Folgen über die Unmöglichkeit effektiver Tests; unserem Zugang am nächsten steht die dortige Idee, Zufälligkeit über das Bestehen effektiver Tests zu fassen, mit dem Unterschied, dass wir endliche Folgen und eine einzige, rein kombinatorische Kenngröße betrachten.

Als experimentelle Quelle dient das Gerät `/dev/random` einer Linux-Installation; seine Arbeitsweise ist dokumentiert und mehrfach analysiert worden [2]. Die hier verwendete Auswertung ist unabhängig von der speziellen Quelle; wir greifen `/dev/random` nur als notorisch guten Lieferanten für Bitfolgen heraus, deren Zustandekommen nicht kausal erklärt werden soll.

Die erkenntnistheoretische Rahmung folgt Popper [8]. Auf die Frage, ob und in welchem Sinne die Klassen B_N den wahrscheinlichkeitstheoretischen Zufallsbegriff ersetzen können, kommen wir in Abschnitt 9 zurück.

3. Grundlagen: Läufe und Teilläufe

Definition 1 (Menge der endlichen Bitfolgen)**.**

$$\mathcal{B} := \{(b_m)_{1 \le m \le M < \infty},\ b_m \in \{0, 1\}\} \tag{1}$$

Wir gehen von einer beliebigen endlichen Bitfolge

$$(b_m)_{1\leq m\leq M,\, M<\infty} \in \mathcal{B} \tag{2}$$

aus. Zum Beispiel

$$10001010100111110000011111110001010101 \tag{3}$$

Diese Bitfolge können wir auch als Folge von Läufen auffassen. Dabei ist ein Lauf eine Folge von gleichbleibenden Bits. In unserem Beispiel sind es

$$1\cdot 000\cdot 1\cdot 0\cdot 1\cdot 0\cdot 1\cdot 00\cdot 11111\cdot 00000\cdot 1111111\cdot 000\cdot 1\cdot 0\cdot 1\cdot 0\cdot 1\cdot 0\cdot 1 \tag{4}$$

Definition 2 (Lauf)**.** Ein *Lauf* einer Bitfolge $(b_m)_{1\leq m\leq M} \in \mathcal{B}$ ist eine konstante, in beiden Richtungen maximale Teilfolge $(b_p, \ldots, b_q)$ mit $1 \leq p \leq q \leq M$:

$$b_p = b_{p+1} = \cdots = b_q, \qquad (p = 1 \text{ oder } b_{p-1} \neq b_p)\,, \qquad (q = M \text{ oder } b_q \neq b_{q+1})\,. \tag{5}$$

Die Zahl $q - p + 1$ heißt die *Länge* des Laufs.

Lemma 3 (Läufzerlegung)**.** *Jede Bitfolge $(b_m)_{1\leq m\leq M} \in \mathcal{B}$ zerfällt eindeutig in Läufe. Mit $m_1 := 1$ und der rekursiven Vorschrift*

$$m_{i+1} := \min\left(\{m > m_i :\ b_m \neq b_{m_i}\} \cup \{M+1\}\right) \tag{6}$$

ergeben sich eindeutige Zerlegungspunkte $1 = m_1 < m_2 < \cdots < m_N < m_{N+1} = M+1$ mit Läufen $(b_{m_i}, \ldots, b_{m_{i+1}-1})$ und Laufängen $r_n := m_{n+1} - m_n$. Insbesondere gilt

$$\sum_{n=1}^{N} r_n = M, \tag{7}$$

und aufeinanderfolgende Läufe tragen komplementäre Bitwerte. Die Folge $(r_n)_{1\leq n\leq N} \in \mathbb{N}$ heißt die Lauffolge *von* (b_m).

Beweis. Da (b_m) auf jedem Block $[m_i, m_{i+1})$ konstant ist und $b_{m_{i+1}} \neq b_{m_i}$ gilt (sofern $m_{i+1} \leq M$), ist (6) wohldefiniert und liefert eine streng wachsende Folge, die nach endlich vielen Schritten $M+1$ erreicht. Die Maximalitätsbedingungen aus (5) sichern, dass jeder Block ein Lauf im Sinne der Definition ist. Die Eindeutigkeit folgt daraus, dass der erste Lauf bei $m = 1$ beginnen muss und jeder Folgelauf durch das Ende des vorherigen festgelegt ist. □

Bemerkung 4 (Umkehrung und Komplement)*.* Jede Bitfolge kann von rechts nach links ebenso wie von links nach rechts gelesen werden, ohne dass sich ihre Lauffolge ändert; ohne Beschränkung der Allgemeinheit lesen wir Folgen daher stets von links nach rechts. Der Beweis ist trivial und wird hier nicht ausgeführt.

Zwei Bitfolgen besitzen genau dann dieselbe Lauffolge, wenn sie gleich oder binäre Komplemente voneinander sind: Die Lauffolge legt – zusammen mit dem Bit des ersten Laufs – wegen des alternierenden Aufbaus die Bitfolge eindeutig fest, und der Übergang zum Komplement ändert nur dieses eine Bit.

Da in sämtlichen Häufigkeitsbetrachtungen dieser Arbeit nur die Längen der Läufe eingehen, identifizieren wir Läufe fortan mit ihrer Länge und arbeiten mit Lauffolgen $(r_n) \in \mathbb{N}$.

Des Weiteren gilt die triviale Überlegung, dass ein Lauf der Länge N Läufe $s < N$ mit entsprechender Häufigkeitsverteilung

$${}_N S = ({}_N s_n)_{0<n\leq N}, \quad \text{mit } {}_N s_n = N - n + 1 \tag{8}$$

beinhaltet. Zur Illustration nehmen wir den Lauf $N = 8$

$$1\,1\,1\,1\,1\,1\,1\,1 \quad \text{bzw.} \quad 0\,0\,0\,0\,0\,0\,0\,0 \tag{9}$$

Dann enthält N:

$$\begin{aligned}
{}_8s_1 &= 8 - 1 + 1 = 8 \text{ Läufe } 1, & {}_8s_2 &= 8 - 2 + 1 = 7 \text{ Läufe } 2,\\
{}_8s_3 &= 8 - 3 + 1 = 6 \text{ Läufe } 3, & {}_8s_4 &= 8 - 4 + 1 = 5 \text{ Läufe } 4,\\
{}_8s_5 &= 8 - 5 + 1 = 4 \text{ Läufe } 5, & {}_8s_6 &= 8 - 6 + 1 = 3 \text{ Läufe } 6,\\
{}_8s_7 &= 8 - 7 + 1 = 2 \text{ Läufe } 7, & {}_8s_8 &= 8 - 8 + 1 = 1 \text{ Lauf } 8.
\end{aligned}$$

Definition 5 (Teillauf)**.** Die in einem Lauf enthaltenen Läufe kleinerer Länge heißen fortan Teilläufe.

4. Der Einschritttest

Wir fassen die zu untersuchende Bitfolge als Folge von Läufen

$$(r_n)_{0<n\leq N}, \quad r_n \in N, \quad N \text{ geeignet} \tag{10}$$

auf. Um eine solche Folge auszuwerten, ermitteln wir die kumulierte Verteilung

$$L = (l_p)_{0<p\leq P<\infty} \tag{11}$$

der Häufigkeiten l_p der Läufe und der darin enthaltenen Teilläufe der Folge $(r_n)_{0<n\leq N<\infty}$.

Wir motivieren diesen Ansatz durch die folgende Überlegung: Für die Ausgabe $(b_m) \in \mathcal{B}$ eines Zufallsalgorithmus, der unabhängige, gleichverteilte Bits liefert, gilt $P(b_{n-1} = b_n) = P(b_{n-1} \neq b_n) = \frac{1}{2}$ [1]. Statt auf Grenzwertaussagen zurückzugreifen, bevorzugen wir es, für jedes Bit b_m aus (b_m) die Häufigkeit der Fälle $(b_m) = (b_{m+i})$ zu betrachten. Dies führt uns zu dem Ansatz der kumulierten Verteilungen: Wir fassen $(b_m) \in \mathcal{B}$ als Folge (r_n) auf und ermitteln die Häufigkeiten ihrer Läufe und der darin enthaltenen Teilläufe.

Trivialerweise ist der größte Index P der Folge (11) durch den längsten Lauf von (r_n) gegeben:

$$P = \max\{r_1, \ldots, r_N\} \tag{12}$$

Aus den vorangegangenen Überlegungen ergibt sich die folgende Vorschrift, die wir hier mit der Bezeichnung Einschritttest versehen.

```
Eingabe: Eine beliebige Folge von Läufen der Länge N, N endlich
Lege einen Array L der Länge maximal N an
fuer jeden Lauf der Folge
    ermittle die Laenge des Laufs (hier l)
    fuer jede Stelle i = 1 bis l
        erhoehe die Haeufigkeit an der Stelle i um l - i + 1
Gib L aus
```

Algorithmus 1: Einschritttest für Folgen von Läufen

Man kann dasselbe Verfahren auf die der Folge von Läufen (r_n) zugrunde liegende Bitfolge (b_m) bzw. $(\overline{b}_m)$ anwenden. Es ergibt sich eine etwas andere Rechenvorschrift; dass sie dasselbe Ergebnis wie Algorithmus 1 liefert, zeigt Lemma 6. Der Name „Einschritttest“ rührt daher, dass diese Auffassung des Algorithmus – die chronologisch erste – die Eingabe Schritt für Schritt abarbeitet; die innere Schleife durchläuft sie für jedes einzelne Bit mit dem jeweils aktuellen Zählerstand z.

```
Eingabe: Eine beliebige endliche Folge von Bits der Laenge P, P
   endlich
Lege einen Array L der Laenge maximal P an
Lege einen Zaehler z mit dem Wert 0 an
fuer jedes Bit der Folge (Schritt fuer Schritt)
    hat sich der Wert des Bits im Vergleich zum vorherigen Schritt
   geaendert?
    wenn nein, dann erhoehe den Wert des Zaehlers z um 1
    wenn ja, dann stelle den Zaehler z auf 1 zurueck
    fuer jede Stelle i = 1 bis z        (fuer jedes einzelne Bit!)
        erhoehe die Haeufigkeit an der Stelle i um 1
Gib L aus
```

Algorithmus 2: Einschritttest für Bitfolgen

Lemma 6 (Äquivalenz)**.** *Die Algorithmen 1 und 2 liefern für jede Bitfolge dieselbe kumulierte Verteilung.*

Beweis. Betrachte einen Lauf der Länge l. Algorithmus 1 erhöht durch diesen Lauf die Häufigkeit an der Stelle i ($1 \le i \le l$) um $l - i + 1$. Algorithmus 2 hat beim j-ten Bit des Laufs den Zählerstand $z = j$; dieses Bit erhöht also die Stellen 1 bis j. Die Stelle i wird daher von den Bits $j = i, \ldots, l$ erhöht, also genau $l - i + 1$-mal. Beide Algorithmen tragen für jeden Lauf denselben Beitrag ein, und die Beiträge verschiedener Läufe addieren sich unabhängig. □

5. Ein motivierendes Experiment

Wir greifen auf ein experimentelles Mittel zurück, um eine Bitfolge zu erzeugen, deren Struktur nicht kausal erklärt werden soll: eine sogenannte Zufallsfolge. Ein solches experimentelles Mittel ist das Gerät `/dev/random` einer beliebigen Linux-Installation [2], das den Ruf besitzt, ein notorisch guter Lieferant solcher Bitfolgen zu sein, auch wenn es relativ langsam arbeitet. Darüber hinaus kann seine Arbeitsweise lückenlos nachgeprüft werden. Wir sammeln die Ausgabe von `/dev/random` über einen Zeitraum von etwa fünfzehn Monaten (13. November 2016 bis 15. Februar 2018) mit einem Hintergrundskript, das permanent läuft: Sobald 1024 Bytes aus `/dev/random` gelesen sind, wird die Datei geschrieben und der nächste Block unmittelbar danach gelesen.

```
dd if=/dev/random of="$(date +%Y_%m_%d_%H_%M_%S)" bs=1024 count=1
```

Das Skript ist eine Endlosschleife, die im Hintergrund läuft und bei jedem Systemstart automatisch gestartet wird. (Kurze Lesezugriffe des Entropie-Pools ergeben gelegentlich Dateien unter 1024 Bytes.) Das Ergebnis sind 9 856 Dateien mit zusammen 9 793 146 Bytes.[1] Das Ergebnis dieses experimentellen Vorgehens ist eine endliche Bitfolge, deren Länge M durch das Experiment bedingt ist:

$$(b_m)_{1 \le m \le M < \infty}, \quad b_m \in \{0, 1\}, \tag{13}$$

mit $M = 78\,345\,168$ Bits.
Wie in Abschnitt 3 über Läufe und Teilläufe ausgeführt, fassen wir nun die Folge (b_m) aus (13) als eine Folge gemäß (10) auf und werten sie mit dem Einschritttest aus. Das Ergebnis der Auswertung

[1] Nach Definition des Einschritttests zählt l_1 jedes Bit der Folge genau einmal; die Konsistenzbedingung $l_1 = M = 78\,345\,168$ ist in der Auswertung automatisch geprüft. Der Datenbestand sowie das Erfassungs- und das Auswertungsskript sind im Repository enthalten (`dev/random` bzw. `paper/experiment`).

der Ausgabe des Geräts `/dev/random` ist in Tabelle 1 protokolliert. Es fällt sofort auf, dass die Verteilung der Läufe und Teilläufe der Ausgabe des Geräts `/dev/random` exponentiell fallend verläuft: $\log_2 l_n$ nimmt mit wachsendem n um nahezu exakt 1 pro Schritt ab. Wir stellen diese Ergebnisse in Abbildung 1 graphisch dar; die dort und in Tabelle 1 ausgewiesene theoretische Reihe ist die Erwartung $l_n = M \cdot 2^{1-n}$ bei unabhängigen, gleich verteilten Bits. Ein Chi-Quadrat-Anpassungstest der Verteilung der Lauflängen gegen die geometrische Verteilung ergibt $\chi^2 = 19{,}8$ bei 22 Freiheitsgraden (kritischer Wert zum Niveau 5%: 33,9); die Schätzung der Übergangswahrscheinlichkeit $\hat{p} = 0{,}49994 \pm 0{,}00006$ liegt um 1,05 Standardabweichungen unter $\frac{1}{2}$. Die Erhebung ist also mit unabhängigen, gleichverteilten Bits vereinbar. Im Sinne der intersubjektiven Nachprüfung dieses Experimentes wird der Leser ausdrücklich dazu ermutigt, dieses Experiment mit den Mitteln seiner Wahl nachzustellen.

Länge	Häufigkeit	$\log_2$	Erwartung	Länge	Häufigkeit	$\log_2$	Erwartung
1	78345168	26,22	78345168	13	19282	14,23	19127
2	39177249	25,22	39172584	14	9588	13,23	9564
3	19592313	24,22	19586292	15	4753	12,21	4782
4	9798825	23,22	9793146	16	2364	11,21	2391
5	4900914	22,22	4896573	17	1174	10,20	1195
6	2451773	21,23	2448287	18	561	9,13	598
7	1227083	20,23	1224143	19	248	7,95	299
8	614018	19,23	612072	20	110	6,78	149
9	307704	18,23	306036	21	50	5,64	75
10	153948	17,23	153018	22	22	4,46	37
11	76935	16,23	76509	23	11	3,46	19
12	38572	15,24	38254	24	5	2,32	9

Tabelle 1: Häufigkeit l_n der Läufe und Teilläufe der Ausgabe von `/dev/random`, berechnet aus dem vollständigen Datenbestand ($M = 78\,345\,168$ Bits in 9 856 Dateien; der längste Lauf hat die Länge 25). Die dritte Spalte gibt $\log_2 l_n$, die vierte die Erwartung $M \cdot 2^{1-n}$ (gerundet) bei unabhängigen, gleich verteilten Bits.

Diese Beobachtung lässt vermuten, dass die Ausgabe des Geräts `/dev/random` – oder irgendeines Geräts, das den Zweck erfüllt, Bitfolgen ohne kausale Erzeugung zu liefern – einen annähernd exponentiell fallenden Verlauf der Verteilung ihrer Läufe und enthaltenen Teilläufe aufweist. Widersprüchlicherweise wäre damit doch eine Struktur des Zufalls erklärt. Wir widmen uns in den nächsten Abschnitten der Untersuchung solcher Bitfolgen, ohne Rücksicht auf die Art, wie sie gewonnen werden.

6. Bitfolgen mit exponentiell kumulierten Häufigkeitsverteilungen

6.1. Kumulierte Verteilung und die Klassen B_N

Definition 7 (Kumulierte Verteilung)**.** Sei $({}_N b_m)$ eine Bitfolge aus $\mathcal{B}$ und $(r_n)_{0<n\leq N<\infty}$ ihre entsprechende Folge von Läufen. Wir bezeichnen mit der Folge

$$ {}_N L := ({}_N l_i)_{0<i\leq N<\infty} \tag{14}$$

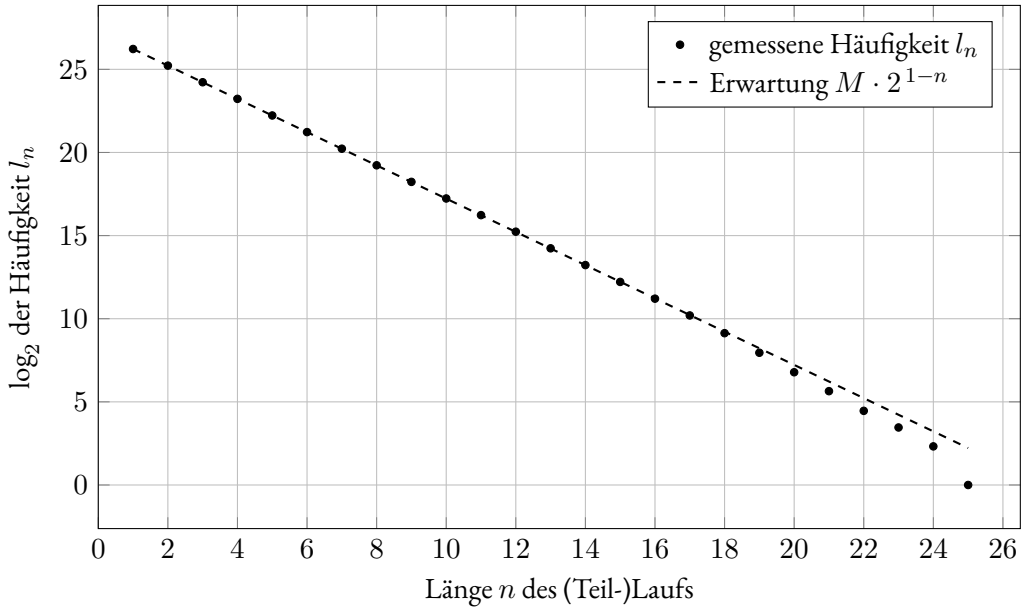


Abbildung 1: Logarithmische Darstellung der Ergebnisse aus Tabelle 1: $\log_2 l_n$ gegen n (Punkte) samt Erwartung $M \cdot 2^{1-n}$ (gestrichelt).

die Verteilung der Häufigkeiten der Läufe und enthaltenen Teilläufe der Bitfolge $({}_N b_m)$ und nennen sie die kumulierte Verteilung.

Definition 8 (Exponentiell fallende kumulierte Verteilung)**.** Unter der exponentiell fallenden, kumulierten Verteilung ${}_N L := ({}_N l_i)_{0<i\leq N<\infty}$ einer Bitfolge $({}_N b_m) \in \mathcal{B}$ verstehen wir

$$\forall\, 0 < n \leq N < \infty : \quad {}_N l_n = 2^{N-n} \tag{15}$$

Äquivalent dazu ist

$${}_N l_N = 1 \tag{16}$$

$${}_N l_{n-1} = 2\, {}_N l_n, \quad 2 \leq n \leq N \tag{17}$$

Wir machen darauf aufmerksam, dass die kumulierte Verteilung einer Bitfolge nicht hinreicht, um diese Folge vollständig zu charakterisieren. Vielmehr entspricht diese Verteilung einer Klasse von Bitfolgen. Daher die folgende Definition.

Definition 9 (Klassen B_N)**.** Wir bezeichnen mit B_N die Menge der Folgen mit längstem Lauf N und mit exponentiell fallender, kumulierter Verteilung gemäß (15) bzw. (16) und (17).

Definition 10 (Menge der Klassen)**.** Menge der Klassen der Bitfolgen mit exponentiell fallenden, kumulierten Verteilungen:

$$\mathcal{B}_{\mathcal{Z}} := \{B_N,\, 0 < N < \infty\} \tag{18}$$

6.2. Die Grundverteilung

Nachdem die grundlegenden Definitionen gestellt wurden, können wir uns der Untersuchung der Elemente aus einem beliebigen B_N widmen. Aus der Definition der exponentiell fallenden, kumulierten Verteilung ausgehend kann untersucht werden, ob die Verteilung der Häufigkeiten ihrer Läufe ohne Teilläufe einer Systematik unterliegt. Wir rechnen also aus jeder Häufigkeit der kumulierten Verteilung ${}_N L$ ihre bereits abgegoltenen Teilläufe heraus; das Ergebnis nennen wir die Grundverteilung.

Definition 11 (Grundverteilung)**.** Sei $(b_m) \in \mathcal{B}$ eine Bitfolge mit Lauffolge $(r_n)_{1\le n\le R}$ und kumulierter Verteilung $L = (l_p)_{1\le p\le P}$. Die *Grundverteilung* $G = (g_p)_{1\le p\le P}$ zählt die Läufe je Länge,

$$g_p := \left|\{n \in \{1,\dots,R\} : r_n = p\}\right|, \qquad 1 \le p \le P. \tag{19}$$

Kumulierte und Grundverteilung hängen über die Dreiecksbeziehung

$$l_p = \sum_{k=p}^{P} (k-p+1)\cdot g_k, \qquad 1 \le p \le P, \tag{20}$$

zusammen: Jeder Lauf der Länge k enthält genau $k-p+1$ Teilläufe der Länge p. Wegen des Dreiecksaufbaus von (20) ist G durch L eindeutig bestimmt; die rekursive Auflösung von oben,

$$g_P = l_P, \qquad g_p = l_p - \sum_{k=p+1}^{P} (k-p+1)\cdot g_k \quad (p = P-1,\dots,1), \tag{21}$$

ist genau das im Folgenden beschriebene Abgeltungsverfahren.

Zur Veranschaulichung vollziehen wir die Auflösung (21) an einem Beispiel aus B_5 nach. In diesem Fall betragen die Werte von ${}_5L = ({}_5l_i)_{0<i\le 5}$

$${}_5l_i = 2^{5-i} \tag{22}$$

Das heißt explizit:

- ${}_5l_1 = 16$,
- ${}_5l_2 = 8$,
- ${}_5l_3 = 4$,
- ${}_5l_4 = 2$,
- ${}_5l_5 = 1$.

Der längste Lauf beträgt 5 und ist kein Teillauf. Wir ziehen also die durch den Lauf der Länge 5 abgegoltenen Teilläufe ab und erhalten:

- 5 Teilläufe 1,
- 4 Teilläufe 2,
- 3 Teilläufe 3,
- 2 Teilläufe 4,
- 1 Lauf 5.

Nun ist der Lauf 5 abgegolten; damit sind auch die beiden Läufe 4 abgegolten. Noch geltend zu machen sind die folgenden Läufe:

- ${}_5l_1 - 5 = 11$ Läufe 1,
- ${}_5l_2 - 4 = 4$ Läufe 2,

- ${}_5l_3 - 3 = 1$ Läufe 3,
- ${}_5l_4 - 2 = 0$ Läufe 4,
- ${}_5l_5 - 1 = 0$ Läufe 5.

Wir machen nun den Lauf 3 geltend, da dieser nicht mehr als Teillauf vorhanden ist. Das heißt

- 3 Teilläufe 1,
- 2 Teilläufe 2,
- 1 Lauf 3.

Nun ist der Lauf 3 abgegolten. Noch geltend zu machen sind die folgenden Läufe:

- ${}_5l_1 - 5 - 3 = 8$ Läufe 1,
- ${}_5l_2 - 4 - 2 = 2$ Läufe 2,
- ${}_5l_3 - 3 - 1 = 0$ Läufe 3.

Wir machen nun die Läufe 2 geltend. Das heißt

- 4 Teilläufe 1,
- 2 Teilläufe 2.

Nun ist der Lauf 2 abgegolten. Noch geltend zu machen sind die folgenden Läufe:

- ${}_5l_1 - 5 - 3 - 4 = 4$ Läufe 1.

Da die Läufe 1 keine Teilläufe enthalten, werden sie unmittelbar abgegolten. Das Ergebnis der Auflösung (21) ist die Grundverteilung

$$ {}_5G = (\ 4; 2; 1; 0; 1\ ). \tag{23}$$

Eine mögliche Realisierung dieser Verteilung ist

$$(\ 0101001100011111\ ) \text{ mit } (\ \underbrace{0\,1\,0\,1}_{4}\ \underbrace{00\,11}_{2}\ \underbrace{000}_{1}\ \underbrace{\ }_{0}\ \underbrace{11111}_{1}\ ) \in B_5 \tag{24}$$

Natürlich ist jede andere Permutation dieser Folge von Läufen gültig. Dabei werden bei Bedarf die Werte der Bits komplementiert. Wir wiederholen beispielhaft diese Berechnung für Bitfolgen aus den Klassen B_1 bis B_8:

$$\begin{aligned}
(\ 1\ )_{[B_1]} \\
(\ 0\ ;\ 1\ )_{[B_2]} \\
(\ 1\ ;\ 0\ ;\ 1\ )_{[B_3]} \\
(\ 2\ ;\ 1\ ;\ 0\ ;\ 1\ )_{[B_4]} \\
(\ 4\ ;\ 2\ ;\ 1\ ;\ 0\ ;\ 1\ )_{[B_5]} \\
(\ 8\ ;\ 4\ ;\ 2\ ;\ 1\ ;\ 0\ ;\ 1\ )_{[B_6]} \\
(\ 16\ ;\ 8\ ;\ 4\ ;\ 2\ ;\ 1\ ;\ 0\ ;\ 1\ )_{[B_7]} \\
(\ 32\ ;\ 16\ ;\ 8\ ;\ 4\ ;\ 2\ ;\ 1\ ;\ 0\ ;\ 1\ )_{[B_8]}
\end{aligned} \tag{25, 26}$$

Wir bemerken sofort, dass die Grundverteilungen der Elemente aus den B_N mit wachsendem N eine einfache Struktur aufweisen: Die Häufigkeiten der Läufe 2 bis N verschieben sich um eine Stelle nach rechts, und die Häufigkeit der Läufe 1 verdoppelt sich. Wir wollen diese experimentelle Beobachtung formal untermauern.

Satz 12. *Sei $N \geq 2$ und $({}_N b_m) \in B_N$. Dann gilt für die Grundverteilung $({}_N g_n)_{1 \leq n \leq N}$ der Läufe von $({}_N b_m)$*

$$ {}_N g_n = 2^{N-2-n}, \quad 1 \leq n \leq N-2, \tag{27}$$

$$ {}_N g_{N-1} = 0, \tag{28}$$

$$ {}_N g_N = 1. \tag{29}$$

Insbesondere erfüllt ${}_N G$ die Rekursion ${}_N g_n = 2\,{}_N g_{n+1}$ für $1 \leq n \leq N-3$. (Für $N = 1$ gilt ${}_1 g_1 = 1$.)

Beweis. Da der längste Lauf die Länge N hat, ist $P = N$, und es ist ${}_N l_n = 2^{N-n}$ für alle $1 \leq n \leq N$ (Gleichung (15)). Wir lösen die Rekursion (21) von oben nach unten auf.

Für $n = N$ ist ${}_N g_N =_N l_N = 2^0 = 1$, das ist (29). Für $n = N-1$ ist

$$ {}_N g_{N-1} =_N l_{N-1} - 2 \cdot_N g_N = 2 - 2 = 0, $$

das ist (28).

Für die verbleibenden Indizes zeigen wir die geschlossene Form (27) durch Abwärtsinduktion über $n = N-2, \ldots, 1$. Subtraktion der Gleichungen (20) für n und $n+1$ liefert dabei

$$ {}_N l_n - {}_N l_{n+1} = {}_N g_n + \sum_{k=n+1}^{N} {}_N g_k, \qquad 1 \leq n \leq N-1, \tag{30}$$

denn der Beitrag eines Laufs der Länge $k > n$ an l_n übersteigt seinen Beitrag an l_{n+1} um genau 1.

Induktionsanfang $n = N-2$: Aus (21) folgt

$$ {}_N g_{N-2} =_N l_{N-2} - 2 \cdot \underbrace{{}_N g_{N-1}}_{=0} - 3 \cdot \underbrace{{}_N g_N}_{=1} = 4 - 3 = 1 = 2^{N-2-(N-2)}. $$

Induktionsschritt: Sei $n \leq N-3$ und es gelte bereits ${}_N g_k = 2^{N-2-k}$ für alle $n < k \leq N-2$. Aus (30) und ${}_N l_n - {}_N l_{n+1} = 2^{N-n-1}$ folgt dann mit der geometrischen Summe

$$ {}_N g_n = 2^{N-n-1} - \sum_{k=n+1}^{N-2} 2^{N-2-k} - \underbrace{{}_N g_{N-1}}_{=0} - \underbrace{{}_N g_N}_{=1} = 2^{N-n-1} - \left(2^{N-2-n} - 1\right) - 1 = 2^{N-2-n}. $$

Das zeigt (27) für alle $1 \leq n \leq N-2$ und damit die Behauptung. □

Bemerkung 13 (Besonderheit des Verfahrens). In keinem Element aus B_N, $N \geq 2$, tritt ein Lauf der Länge $N-1$ auf: ${}_N g_{N-1} = 0$. Die Grundverteilung springt unmittelbar von ${}_N g_{N-2} = 1$ auf ${}_N g_N = 1$. Das ist keine Unregelmäßigkeit, sondern eine besondere, systematische Eigenschaft des Verfahrens: Der Häufigkeitsanteil der Länge $N-1$ ist in der Kumulation vollständig in den Teilläufen des einzigen Laufs der Länge N aufgegangen – ${}_N l_{N-1} = 2$ zählt genau dessen beide Teilläufe der Länge $N-1$ –, und bei der Auflösung von oben wird dieser Anteil abgezogen, bevor ein eigener Lauf der Länge $N-1$ frei werden könnte.

Definition 14 (Summe T_m)**.** Für alle $m \geq 0$ sei

$$ T_m := \sum_{k=1}^{m} k \cdot 2^{m-k}. \tag{31}$$

Lemma 15 (Summenidentität). *Für alle $m \geq 0$ gilt*

$$T_m = 2^{m+1} - m - 2. \tag{32}$$

Beweis. Induktion über m. Für $m = 0$ ist die Summe leer und $T_0 = 0 = 2^1 - 0 - 2$. Im Schritt gilt

$$T_{m+1} = \sum_{k=1}^{m+1} k{\cdot}2^{m+1-k} = 2\sum_{k=1}^{m} k{\cdot}2^{m-k} + (m+1) = 2T_m + (m+1) = 2^{m+2} - (m+1) - 2. \quad \square$$

Korollar 16 (Umkehrung und Existenz). *Besitzt eine Bitfolge die Laufverteilung*

$${}_N g_n = 2^{N-2-n}\ (1 \leq n \leq N-2), \qquad {}_N g_{N-1} = 0, \qquad {}_N g_N = 1, \tag{33}$$

so liegt sie in B_N. Insbesondere ist B_N für jedes $N \geq 1$ nichtleer.

Beweis. Zu zeigen ist, dass die kumulierte Verteilung ${}_N l_n = 2^{N-n}$ beträgt. Aus (20) und (33) folgt für $1 \leq n \leq N-2$ mit $m := N-2-n$

$$\begin{aligned} {}_N l_n &= \sum_{k=n}^{N-2} (k-n+1) \cdot 2^{N-2-k} + (N-n+1) \cdot 1 \\ &= \sum_{j=0}^{m} (j+1) \cdot 2^{m-j} + (m+3) = \sum_{k=1}^{m+1} k \cdot 2^{m+1-k} + (m+3) \\ &= \big(2^{m+2} - m - 3\big) + (m+3) = 2^{m+2} = 2^{N-n}, \end{aligned}$$

wobei der vorletzte Schritt die Summenidentität (Lemma 15, $m \mapsto m+1$) benutzt. Für $n = N-1$ ist ${}_N l_{N-1} = 2 \cdot_N g_N = 2 = 2^{N-(N-1)}$ und für $n = N$ ist ${}_N l_N =_N g_N = 1$. Der längste Lauf hat die Länge N, also liegt die Bitfolge in B_N. $\square$

6.3. Die Abgeltung der Teilläufe

Die Abgeltung von Teilläufen, wie sie in dem Verfahren zur Herleitung der Grundverteilung dargestellt wurde, kann auch direkt berechnet werden: Die Häufigkeit ${}_N t_n$ der abgegoltenen Teilläufe der Länge n ist die Summe der Teillauf-Beiträge aller längeren Läufe; ein Lauf der Länge i enthält genau $i - n + 1$ Teilläufe der Länge n:

$${}_N t_n = \sum_{i=n+1}^{N} (i-n+1) \cdot_N g_i, \qquad 1 \leq n \leq N. \tag{34}$$

Mit ${}_N t_n =_N l_n -_N g_n$ ergibt sich der allgemeine Ausdruck

$${}_N l_n -_N g_n = \sum_{i=n+1}^{N} (i-n+1) \cdot_N g_i, \tag{35}$$

der für beliebige Grundverteilungen gilt und die Dreiecksbeziehung (20) äquivalent umformt. Für die exponentiell abfallenden kumulierten Verteilungen ${}_N l_n = 2^{N-n}$ und die Grundverteilung aus Satz 12 rechnen wir drei Fälle nach.

♦ $n = N$: Die Summe in (34) ist leer, also

$${}_N t_N = 0. \tag{36}$$

♦ $n = N - 1$:

$$ {}_Nt_{N-1} = (N - (N-1) + 1) \cdot \underbrace{{}_Ng_N}_{=1} = 2 = 2^{N-(N-1)} - \underbrace{{}_Ng_{N-1}}_{=0}. \tag{37} $$

♦ $1 \leq n \leq N-2$: Mit ${}_Nl_n = 2^{N-n}$ und ${}_Ng_n = 2^{N-2-n}$ gilt

$$ {}_Nt_n = {}_N\, l_n - {}_N\, g_n = 2^{N-n} - 2^{N-2-n} = 3 \cdot 2^{N-2-n}. \tag{38} $$

Aus (38) ergibt sich nach Indexverschiebung die Verschiebungsstruktur, die auch in den Beispielen (25) bis (26) sichtbar ist: Für $1 \leq n \leq N-2$ gilt

$$ {}_Nt_n = 3 \cdot 2^{N-2-n} = {}_{N-n+1}t_1, \tag{39} $$

denn nach (38) ist ${}_Mt_1 = 3 \cdot 2^{M-3}$, also für $M = N - n + 1$ gerade die rechte Seite von (39). Die abgegoltenen Teilläufe der Länge n einer Klasse B_N stimmen also mit den abgegoltenen Teilläufen der Länge 1 der Klasse B_{N-n+1} überein.

Zusammenfassend notieren wir:

Lemma 17. *Sei $N < \infty$. Dann gilt*

$$ 2^N = (N+1) + \left(1_{\{N>1\}} \cdot \sum_{n=0}^{N-2} (N-1-n) \cdot 2^n \right) \tag{40} $$

Beweis. Die Substitution $k = N - 1 - n$ führt die Summe über in

$$ \sum_{n=0}^{N-2} (N-1-n) \cdot 2^n = \sum_{k=1}^{N-1} k \cdot 2^{N-1-k} = T_{N-1}, $$

und mit dem Lemma 15 ($m = N-1$) ist $T_{N-1} = 2^N - N - 1$, also

$$ (N+1) + T_{N-1} = (N+1) + \left(2^N - N - 1\right) = 2^N. \qquad \square $$

Bemerkung 18 (Gesamtlänge). Die rechte Seite von (40) zerlegt die Bit-Länge eines Vertreters von B_{N+1} nach Beiträgen der einzelnen Läufe: Der einzige Lauf der Länge $N+1$ trägt $N+1$ Bits bei, ein Lauf der Länge $k \leq N-1$ kommt 2^{N-1-k}-mal vor und trägt jeweils k Bits bei; die Länge N trägt nichts bei, denn ${}_{N+1}g_N = 0$ (Bemerkung 13). Das Lemma besagt also: Jeder Vertreter von B_{N+1} besitzt genau 2^N Bits, jeder Vertreter von B_N dementsprechend genau 2^{N-1} Bits.

7. Algorithmen zur Erzeugung eines Elementes aus B_N

Motiviert durch die Ergebnisse aus dem vorherigen Abschnitt geben wir einen iterativen Algorithmus zur Berechnung eines Klassenvertreters der B_N mit $N < \infty$ an.
Die Richtigkeit des Algorithmus ergibt sich unmittelbar aus Korollar 16: Er erzeugt genau die Laufverteilung (33), und jede Bitfolge mit dieser Laufverteilung liegt in B_N.
Für den rekursiven Algorithmus nutzen wir die folgende Konstruktion, die den Übergang von B_{N-1} nach B_N beschreibt.

Lemma 19 (Konstruktion)**.** *Sei $N \geq 3$ und $({}_{N-1}b_m) \in B_{N-1}$. Erweitere jeden Lauf von $({}_{N-1}b_m)$ um genau ein Bit und füge 2^{N-3} Läufe der Länge 1 an Stellen ein, an denen der alternierende Aufbau erhalten bleibt (etwa je einen zwischen benachbarte Läufe und einen an ein Ende). Dann liegt das Ergebnis in B_N.*

```
Eingabe: N, die Laenge des laengsten Laufs
Lege einen leeren Bitvektor B an
Haenge einen Lauf der Laenge N an
fuer alle i von N-2 bis 1
    haenge 2^(N-i-2) Laeufe der Laenge i an B an
Gib B aus
```

Algorithmus 3: Iterative Erzeugung einer Folge aus B_N fuer ein beliebiges N

Beweis. Nach Satz 12, angewandt auf B_{N-1}, besitzt $({}_{N-1}b_m)$

$$\sum_{n=1}^{N-3} 2^{N-3-n} + 1 = \left(2^{N-3} - 1\right) + 1 = 2^{N-3}$$

Läufe. Nach der Erweiterung besitzt die Folge die Laufängen $k+1$ anstelle von k sowie 2^{N-3} neue Läufe der Länge 1. Ein Lauf der Länge k enthält $k-n+1$ Teilläufe der Länge n; nach der Erweiterung also $(k+1) - n + 1 = k - (n-1) + 1$. Für die kumulierte Verteilung L' der erweiterten Folge gilt daher für alle $2 \le n \le N$

$$l'_n = \sum_{k \ge n-1} (k - (n-1) + 1) \cdot {}_{N-1}g_k = {}_{N-1}l_{n-1} = 2^{(N-1)-(n-1)} = 2^{N-n},$$

die Erweiterung verschiebt also die kumulierte Verteilung um eine Stelle. Die neuen Läufe der Länge 1 verändern nur l'_1: Nach Bemerkung 18 hat $({}_{N-1}b_m)$ 2^{N-2} Bits, die Erweiterung fügt 2^{N-3} Bits hinzu, die neuen Läufe weitere 2^{N-3} Bits, also ist

$$l'_1 = 2^{N-2} + 2^{N-3} + 2^{N-3} = 2^{N-1}.$$

Der längste Lauf hat die Länge N (Erweiterung des Laufs der Länge $N-1$), und es gilt $l'_n = 2^{N-n}$ für alle $1 \le n \le N$; das Ergebnis liegt also in B_N. □

Am Beispiel $(0001) \in B_3$: Die Erweiterung aller Läufe ergibt $(0000\,11)$ mit Läufen der Längen 4 und 2; die Ergänzung von $2^{4-3} = 2$ Läufen der Länge 1, etwa durch Anhängen zu $(0000\,11\,0\,1)$, liefert die Grundverteilung $(2; 1; 0; 1)$ von B_4.
Das Lemma 19 begründet die Richtigkeit des folgenden rekursiven Algorithmus zur Berechnung eines Klassenvertreters der B_N mit $N < \infty$.

```
Eingabe: Ein Element aus B_N mit N > 2
wiederhole fuer jeden Lauf
    ermittle die Laenge des Laufs und erweitere ihn um 1 Bit
    falls der Lauf die Laenge 1 hatte, fuege 2 Laeufe der Laenge 1
   hinzu
Gib das Ergebnis aus
```

Algorithmus 4: Rekursive Berechnung einer Bitfolge aus einem B_N

Die Fälle $N = 1$ und $N = 2$ können in einer Implementierung des Algorithmus gesondert behandelt werden; eine getestete Referenzimplementierung aller Algorithmen ist in Anhang A angegeben.

8. Zusammengesetzte Elemente aus verschiedenen B_N

Sei nun $B \in \mathbb{N}$ beliebig mit seiner eindeutig bestimmten, binären Darstellung

$$B = \sum_{n=1}^{N} b_n \cdot 2^{n-1}, \quad b_n \in \{0,1\},\ b_N = 1, \tag{41}$$

wobei b_1 das Bit mit dem kleinsten Stellenwert ist. Nach Bemerkung 18 besitzt jeder Vertreter von B_n genau 2^{n-1} Bits. Ordnen wir also dem Bit b_n die Klasse B_n zu, so entspricht die Zahl B genau der Zusammensetzung der Klassen $\{B_n : \ b_n = 1\}$, und die zusammengesetzte Bitfolge hat die durch B vorgegebene Länge:

$$B = \sum_{n=1}^{N} b_n \cdot 2^{n-1} = \sum_{\substack{1 \le n \le N \\ b_n = 1}} |\text{Vertreter von } B_n| \,. \tag{42}$$

Damit können wir auf eindeutige Weise eine beliebige natürliche Zahl B mit einer durch diese Zahl eindeutig bestimmten Zusammensetzung paarweise verschiedener Klassen B_N aus $\mathcal{B}_\mathcal{Z}$ identifizieren und erhalten die zusammengesetzte Häufigkeitsverteilung ${}_B H_N$ der Läufe einer Bitfolge mit der vorgegebenen Länge B. Wir vereinbaren die folgende Definition.

Definition 20 (Zufällige Bitfolge)**.** Jede Bitfolge (b_n), deren Häufigkeitsverteilung ihrer Läufe eine beliebige Zusammensetzung von Häufigkeitsverteilungen paarweise verschiedener Klassen $B_N \in \mathcal{B}_\mathcal{Z}$ ist, nennen wir zufällig.

Bemerkung 21 (Einordnung). Diese Definition ist als Vorschlag zu lesen: Sie ersetzt keine Grenzwert- oder Axiomatik-Aussagen, sondern legt für endliche Folgen eine rein kombinatorische Bedingung fest. Ihr Verhältnis zum wahrscheinlichkeitstheoretischen und zum algorithmischen Zufallsbegriff diskutieren wir in Abschnitt 9.

Die Häufigkeit der Lauflänge n einer beliebigen Zusammensetzung paarweise verschiedener Klassen aus $\mathcal{B}_\mathcal{Z}$ ergibt sich aus Satz 12: Die Klasse B_n trägt (für $b_n = 1$) einen einzigen Lauf der Länge n bei, die Klasse B_{n+1} keinen (denn ${}_{n+1}g_n = 0$), und jede Klasse B_i mit $i \ge n+2$ trägt $2^{\,i-n-2}$ Läufe der Länge n bei:

$${}_B H_n = b_n + \sum_{i=n+2}^{N} b_i \cdot 2^{\,i-n-2}, \qquad 1 \le n \le N. \tag{43}$$

Die Länge der auf diese Art gewonnenen Bitfolge ist damit

$$\begin{aligned} \sum_{n=1}^{N} n \cdot_B H_n &= \sum_{n=1}^{N} n \cdot b_n + \sum_{n=1}^{N} \sum_{i=n+2}^{N} n \cdot b_i \cdot 2^{\,i-n-2} \\ &= \sum_{i=1}^{N} b_i \left(i + \sum_{n=1}^{i-2} n \cdot 2^{\,i-n-2} \right) = \sum_{i=1}^{N} b_i \,(i + T_{i-2}) \\ &= \sum_{i=1}^{N} b_i \cdot 2^{\,i-1} = B, \end{aligned} \tag{44}$$

mit der Summenidentität (Lemma 15, $m = i-2$) im vorletzten Schritt, denn $T_{i-2} = 2^{i-1} - i$. Die Zusammensetzung ist damit konsistent mit der binären Darstellung (41). Wir illustrieren diesen Sachverhalt mit

$$B = 101010101_2 = 341_{10} \tag{45}$$

n	b_n	1	2	3	4	5	6	7	8	9
1	1	1								
2	0	~~0~~	~~1~~							
3	1	1	0	1						
4	0	~~2~~	~~1~~	~~0~~	~~1~~					
5	1	4	2	1	0	1				
6	0	~~8~~	~~4~~	~~2~~	~~1~~	~~0~~	~~1~~			
7	1	16	8	4	2	1	0	1		
8	0	~~32~~	~~16~~	~~8~~	~~4~~	~~2~~	~~1~~	~~0~~	~~1~~	
9	1	64	32	16	8	4	2	1	0	1
	Σ	86	42	22	10	6	2	2	0	1
	n	1	2	3	4	5	6	7	8	9
	Π	86	84	66	40	30	12	14	0	9
Σ	341									

Tabelle 2: Zusammengesetzte Verteilung für $B = 341$: Spaltenweise Beiträge der Klassen B_i mit $b_i = 1$ zur Häufigkeit H_n der Läufe der Länge n; Zeile Σ: H_n, Zeile Π: $n \cdot H_n$. Gestrichene Einträge gehören zu Klassen mit $b_i = 0$.

in der folgenden Tabelle:
Wir sind nun in der Lage, jede beliebige natürliche Zahl einer endlichen, eindeutig bestimmten Teilmenge $\{B_n\}_{n\in\mathbb{N}} \subset \mathcal{B}_\mathcal{Z}$ mit $n_1 \neq n_2\ \forall n_1, n_2$ zuzuordnen und damit eine eindeutig bestimmte, zusammengesetzte Häufigkeitsverteilung von Läufen herzuleiten.
Wir untersuchen nun die logarithmischen Werte der zusammengesetzten Häufigkeiten. Sei

$$C_n := \sum_{i=n}^{N} {}_B H_i \tag{46}$$

die kumulierte Häufigkeit der Läufe der Länge mindestens n der zusammengesetzten Folge; dabei setzen wir $b_j := 0$ für $j > N$. Aus (43) folgt durch Summenvertauschung die Identität

$$C_n = b_n + b_{n+1} + \sum_{j=n+2}^{N} b_j \cdot 2^{\,j-n-1} = \frac{B_n + b_n}{2}, \tag{47}$$

wobei $B_n := \sum_{j=n}^{N} b_j \cdot 2^{\,j-n}$ die natürliche Zahl mit der Binärdarstellung $(b_N \ \ldots \ b_n)$ ist. Da $b_N = 1$, gilt $2^{N-n} \leq B_n \leq 2^{N-n+1} - 1$, also die Band-Aussage

$$2^{N-n-1} \ \leq\ C_n \ \leq\ 2^{N-n}, \qquad \text{d. h.} \qquad N - n - 1 \ \leq\ \log_2 C_n \ \leq\ N - n. \tag{48}$$

Beide Schranken sind scharf: Ist nur b_N gesetzt, so ist $C_n = 2^{N-n-1}$ und $\log_2 C_n = N-n-1$ exakt; sind alle Bits gesetzt, so ist $C_n = 2^{N-n}$. Wird umgekehrt ein einzelnes Bit b_m von 0 auf 1 gesetzt, so wächst C_n für $n \leq m-2$ um $1 + 2^{m-n-1}$ und für $n \in \{m-1, m\}$ um 1; diese Erhöhung ist höchstens C_n selbst (für $n \leq m-2$ wegen $1 + 2^{m-n-1} \leq 2^{N-n-1} \leq C_n$ mit $m \leq N-1$ und $n \leq N-3$, für $n \in \{m-1, m\}$ wegen $C_n \geq 2^{N-n-1} \geq 1$). Der Zweierlogarithmus wächst also um höchstens 1. Das ist der formale Hintergrund der experimentellen Beobachtung aus Abschnitt 5: Dort liegen die logarithmierten Häufigkeiten (dritte Spalte von Tabelle 1) jeweils knapp unter den theoretischen Werten (vierte Spalte).

9. Diskussion

Allein die Häufigkeitsverteilung der Läufe einer Bitfolge entscheidet über ihre Zugehörigkeit zu einem B_N mit $N < \infty$. Dies hat zur Folge, dass nicht nur die scheinbar unregelmäßige Realisierung (24), sondern auch ihre geordneteste Variante 11111 0 111 0 11 0 11 0 in B_5 liegt: Nicht die Anordnung der Läufe, sondern allein ihre Häufigkeitsverteilung entscheidet. Als weitere Illustration können wir jede Bitfolge als binäre Zahl auffassen und mit jeder anderen Bitfolge durch die gewohnten Ordnungsrelationen vergleichen. Wir nehmen die Elemente aus B_5. Das größte Element ist[2] (In der Binärdarstellung steht das höchstwertige Bit links.)

$$11111\,0\,111\,0\,11\,0\,11\,0 \tag{49}$$

Man erkennt sofort, dass die Läufe mit einer Länge größer eins in abfallender Reihe sortiert sind und jeweils von einem Lauf der Länge eins getrennt sind. Das kleinste Element aus B_5 ist dagegen

$$00000\,1\,000\,1\,00\,1\,00\,1 \tag{50}$$

Man erkennt sofort, dass es sich um das binäre Komplement des größten Elementes aus B_5 handelt. Weiter ist

$$1\,00000\,1\,000\,1\,00\,1\,00 \tag{51}$$

das kleinste Element aus B_5, das mit einer Eins anfängt, und sein binäres Komplement

$$0\,11111\,0\,111\,0\,11\,0\,11 \tag{52}$$

das größte Element, das mit einer Null anfängt. Weiterhin gibt es in B_5 kein Element zwischen (51) und (52).
So lässt sich intuitiv eine gewisse Struktur in den B_N erahnen. Für die Zugehörigkeit zu B_N ist diese Wiedererkennbarkeit jedoch ohne Belang; allein die Häufigkeitsverteilung der Läufe entscheidet.

Bemerkung 22 (Kardinalität). Die Klassen B_N sind groß. Ein Element aus B_N ($N \geq 2$) ist durch die Reihenfolge seiner $R = 2^{N-2}$ Läufe und das Bit des ersten Laufs festgelegt (Lemma 3), also gilt mit der Grundverteilung aus Satz 12

$$|B_N| = 2 \cdot \frac{R!}{\prod_{n=1}^{N} g_n!}, \qquad R = 2^{N-2}, \quad g_n = 2^{N-2-n}. \tag{53}$$

(Die Formel ist für $N \leq 5$ durch vollständige Enumeration bestätigt.) Mit der Stirlingschen Formel folgt aus (53)

$$\log_2 |B_N| = 2^{N-1} - \Theta\big(N^2\big)\,. \tag{54}$$

Die Beschreibungsgröße eines Elements aus B_N wächst also im Wesentlichen mit der vollen Bitlänge 2^{N-1} der Folge. Ein Element zu spezifizieren erfordert praktisch die Angabe der gesamten Folge, während die Verifikation der Zugehörigkeit mit dem Einschritttest in linearer Zeit in der Folgenlänge erfolgt. Der Anteil der Klasse an allen Folgen der Länge 2^{N-1} beträgt zwar $2^{-\Theta(N^2)}$ und geht gegen 0 – eine über die triviale Asymmetrie von Spezifikation und Prüfung hinausgehende Komplexitätsaussage, etwa eine Beziehung zu $P \neq NP$ im Sinne kryptographischer Annahmen, lässt sich aus der Kombinatorik der B_N jedoch nicht ableiten.

[2] Die Abstände in der Darstellung dienen lediglich der Lesbarkeit und haben keinerlei Bedeutung für den Inhalt.

Ob die Klassen $\mathcal{B}_\mathcal{Z}$ damit den Zufallsbegriff der Wahrscheinlichkeitstheorie ersetzen können, ist eine philosophische Frage, keine kombinatorische. Die hier vorgeschlagene Definition (Definition 20) ist bewusst rein kombinatorisch und auf endliche Folgen zugeschnitten; sie tritt neben den wahrscheinlichkeitstheoretischen Zufallsbegriff [4] und den algorithmischen Zufallsbegriff von Martin-Löf [7], der Zufälligkeit unendlicher Folgen über das Bestehen sämtlicher effektiver Tests definiert. Vor diesem Hintergrund sei die These erlaubt, dass es nicht den Zufall der Wahrscheinlichkeitstheorie gibt, sondern die Willkür der Wahl eines Elementes aus $\mathcal{B}_\mathcal{Z}$ – und Bemerkung 22 zeigt, dass diese Willkür messbar ist: Sie wächst mit der vollen Länge der gewählten Folge.

10. Zusammenfassung und Ausblick

Wir haben mit dem Einschritttest ein Auswertungsverfahren für endliche Bitfolgen eingeführt, das die Häufigkeiten von Läufen und Teilläufen kumuliert. Die experimentelle Anwendung auf die Ausgabe von `/dev/random` ($78,3$ Mio. Bits, etwa fünfzehn Monate Erhebungsdauer) zeigt einen exponentiell fallenden Verlauf, wie er aus der geometrischen Verteilung der Lauflängen unabhängiger Bits zu erwarten ist; ein Chi-Quadrat-Anpassungstest bestätigt dies. Die Klassen B_N der Bitfolgen mit exakt exponentiell fallender kumulierter Verteilung besitzen eine streng rekursive Grundverteilung (Satz 12); daraus ergeben sich ein iterativer und ein rekursiver Generator, und über die binäre Darstellung natürlicher Zahlen eine eindeutige Zusammensetzung der Klassen (Abschnitt 8).

Eine getestete Referenzimplementierung aller vier Algorithmen samt Mitgliedschaftstest für die Klassen B_N findet sich in Anhang A; die Äquivalenz der beiden Varianten des Einschritttests ist in Lemma 6 bewiesen, das Verhältnis zu den klassischen Zufallsbegriffen und die Kardinalitätsaussagen werden in Abschnitt 9 diskutiert.

Offen und Gegenstand der Weiterarbeit sind:

- die Erweiterung des Experimentes auf weitere Quellen (`/dev/urandom`, Pseudo- und Hardware-Generatoren) und größere Stichproben;
- die englische Fassung des Textes.

A. Referenzimplementierung

Das Python-Modul in Algorithmus 5 implementiert den Einschritttest in beiden Varianten (Algorithmen 1 und 2), die beiden Generatoren (Algorithmen 3 und 4) sowie den Mitgliedschaftstest für die Klassen B_N. Der Selbsttest prüft für $N = 1, \ldots, 16$, dass beide Generatoren nur Folgen mit exakt exponentiell fallender kumulierter Verteilung erzeugen, dass der rekursive Übergang aus B_N nach B_{N+1} führt (Lemma 19), und dass beide Varianten des Einschritttests auf Pseudozufallsbits identische Resultate liefern (Lemma 6).

```
# Referenzimplementierung der Algorithmen aus dem Paper
# "Auswertung endlicher Bitfolgen".
#
#   - einschritt(bits):       Einschritttest, Variante fuer Bitfolgen
#                             (Algorithmus 2)
#   - einschritt_laeufe(r):   Einschritttest, Variante fuer Folgen von
#                             Laeufen (Algorithmus 1)
#   - b_n_iterativ(N):        iterativer Generator fuer B_N
#                             (Algorithmus 3)
#   - b_n_rekursiv(r):        rekursiver Uebergang B_N -> B_{N+1}
#                             (Algorithmus 4)
#   - in_klasse(r, N):        Mitgliedschaftstest fuer B_N
```

```
#
# Alle Funktionen arbeiten auf Folgen von Bits (0/1) bzw. auf
# Folgen von Laufaengen. Rueckgabe jeweils die kumulierte
# Verteilung bzw. die Folge der Laufaengen.


def einschritt(bits):
    """Kumulierte Verteilung der Läufe und Teilläufe einer Bitfolge.

    Zählt bei jedem Bit der Folge alle Stellen 1..z, wobei z die
    Länge des bis zu diesem Bit begonnenen Laufs ist.
    """
    L, z, vorgaenger = [], 0, None
    for b in bits:
        if b == vorgaenger:
            z += 1
        else:
            z = 1
            vorgaenger = b
        for i in range(z):
            if i == len(L):
                L.append(0)
            L[i] += 1
    return L


def einschritt_laeufe(laeufe):
    """Kumulierte Verteilung aus der Folge der Lauflängen.

    Jeder Lauf der Länge l erhöht die Stelle i (1 <= i <= l)
    um l - i + 1.
    """
    L = []
    for l in laeufe:
        while len(L) < l:
            L.append(0)
        for i in range(1, l + 1):
            L[i - 1] += l - i + 1
    return L


def b_n_iterativ(N):
    """Klassenvertreter von B_N als Folge von Lauflängen (
   Algorithmus 3).

    Ein Lauf der Länge N, dann für i = N-2, ..., 1 je 2^(N-i-2)
    Läufe der Länge i.
    """
    laeufe = [N]
    for i in range(N - 2, 0, -1):
        laeufe += [i] * (2 ** (N - i - 2))
    return laeufe


def b_n_rekursiv(laeufe):
```

```
    """Übergang B_N -> B_{N+1} (Algorithmus 4).

    Erweitert jeden Lauf um ein Bit und ergänzt 2^(N-2) Läufe der
    Länge 1 (N = laengster Lauf der Eingabe). Die Fälle N = 1 und
    N = 2 erfordern keine Ergänzung.
    """
    N = max(laeufe)
    ergaenzung = 0 if N < 2 else 2 ** (N - 2)
    return [l + 1 for l in laeufe] + [1] * ergaenzung


def in_klasse(laeufe, N):
    """Mitgliedschaftstest: liegt die Folge der Lauflängen in B_N?

    Prüft, ob die kumulierte Verteilung exakt exponentiell fällt:
    l_n = 2^(N-n) für alle 1 <= n <= N.
    """
    L = einschritt_laeufe(laeufe)
    return len(L) == N and all(
        L[n - 1] == 2 ** (N - n) for n in range(1, N + 1)
    )


def laeufe_zu_bits(laeufe, start=0):
    """Realisierung einer Folge von Lauflängen als Bitfolge."""
    bits, b = [], start
    for l in laeufe:
        bits += [b] * l
        b = 1 - b
    return bits


def laeufe_von(bits):
    """Lauflängen einer Bitfolge."""
    laeufe, z = [], 1
    for i in range(1, len(bits)):
        if bits[i] == bits[i - 1]:
            z += 1
        else:
            laeufe.append(z)
            z = 1
    laeufe.append(z)
    return laeufe


if __name__ == "__main__":
    # Selbsttest: Generatoren erzeugen Elemente aus B_N,
    # beide Varianten des Einschritttests stimmen ueberein.
    import random

    for N in range(1, 17):
        r = b_n_iterativ(N)
        assert in_klasse(r, N), N
        assert einschritt(laeufe_zu_bits(r)) == einschritt_laeufe(r)
    , N
```

```
        if N >= 2:
            assert in_klasse(b_n_rekursiv(r), N + 1), N

    random.seed(0)
    for _ in range(20):
        bits = [random.randint(0, 1) for _ in range(5000)]
        assert einschritt(bits) == einschritt_laeufe(laeufe_von(bits
    ))

    print("Alle Selbsttests bestanden.")
```

Algorithmus 5: einschritttest.py: Referenzimplementierung

Literaturverzeichnis

[1] William Feller. *An Introduction to Probability Theory and Its Applications*. 3. Aufl. Bd. 1. New York: Wiley, 1968.

[2] Zvi Gutterman und Dahlia Malkhi. „Hold the RESQ: Analysis of the Linux Random Number Generator“. In: *Proceedings of the 11th ACM Conference on Computer and Communications Security (CCS)*. New York: ACM, 2004, S. 264–275.

[3] Donald E. Knuth. *The Art of Computer Programming, Volume 2: Seminumerical Algorithms*. 3. Aufl. Reading, MA: Addison-Wesley, 1997.

[4] A. N. Kolmogoroff. *Grundbegriffe der Wahrscheinlichkeitsrechnung*. Berlin: Springer, 1933.

[5] Pierre L'Ecuyer. „Uniform Random Number Generation“. In: *Annals of Operations Research* 53 (1994), S. 77–120.

[6] George Marsaglia. *DIEHARD: A Battery of Tests of Randomness*. Techn. Ber. Tallahassee, FL: Florida State University, 1996.

[7] Per Martin-Löf. „The Definition of Random Sequences“. In: *Information and Control* 9.6 (1966), S. 602–619.

[8] Karl R. Popper. *Logik der Forschung*. Wien: Julius Springer, 1935.

[9] Andrew Rukhin u. a. *A Statistical Test Suite for Random and Randomized Cryptographic Algorithms*. NIST Special Publication 800-22 Rev. 1a. National Institute of Standards und Technology, 2010.

[10] A. Wald und J. Wolfowitz. „On a test whether two samples are from the same population“. In: *The Annals of Mathematical Statistics* 11.2 (1940), S. 147–162.